

Benchmarking GUI Agents in Software Building Task

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

2 MOTIVATION

Developers need to compile open source project. However, not all open source projects are easy to install locally.

3 EMPIRICAL STUDY

We conduct an empirical study to summarize common software build errors, that blocked developers.

- *lack of dependency.*
- *wrong configuration.*
- *XXX.*

4 BENCHMARK

We construct a comprehensive benchmark that systematically evaluates GUI agents on real-world deployment scenarios. Our benchmark emphasizes realistic error conditions, diverse technology stacks, execution-based validation, and controllable environments.

4.1 Environment Setup

4.1.1 Operating System Environments. Our benchmark supports three major operating systems: Ubuntu 22.04 LTS, Windows 11, and macOS 13+.The environment provides:

- **Virtual machine isolation:** Each task executes in a clean VM snapshot to ensure independence and reproducibility
- **Full system access:** Agents have complete control over GUI interactions (mouse, keyboard) and terminal operations
- **Pre-configured software:** Common development tools, package managers, and frameworks pre-installed with standard configurations
- **Network access:** Controlled internet connectivity for downloading dependencies and accessing documentation

4.1.2 Software and Tool Ecosystem. We provide a diverse set of pre-installed software representing real development environments:

- **Package managers:** apt, npm, pip, Maven, Gradle, Docker
- **Web servers:** Nginx, Apache HTTP Server
- **Databases:** MySQL, PostgreSQL, MongoDB, Redis
- **Runtime environments:** Node.js, Python, Java JDK, Ruby
- **Development tools:** Git, VS Code, terminal emulators
- **System utilities:** File managers, text editors, process monitors

Each environment is configured with default settings that match typical production deployments, ensuring that errors encountered are representative of real-world scenarios.

4.2 Data Collection Methodology

We collected deployment error scenarios through five complementary approaches to ensure diversity and real-world relevance:

Open-Source Project Analysis (42.1%). We systematically analyzed 250+ popular repositories across different technology stacks, including web applications, microservices, data processing pipelines, and machine learning systems. For each project, we attempted fresh deployment in clean environments and documented naturally occurring errors, dependency conflicts, configuration issues, and version compatibility problems. This yielded 205 scenarios with authentic error patterns.

Issue Tracker Mining (28.7%). We mined GitHub Issues and Stack Overflow questions related to deployment failures, filtering for issues tagged with deployment keywords, questions with accepted answers and high vote counts, and recent issues (2022-2024) to ensure relevance. This process identified 140 common deployment challenges with verified solutions.

Developer Survey (12.3%). We surveyed 45 software developers (experience range: 2-10 years) about their most challenging deployment experiences, collecting common error patterns, time-consuming debugging scenarios, platform-specific challenges, and multi-component deployment issues. This contributed 60 diverse real-world scenarios.

Documentation Analysis (9.6%). We reviewed official documentation, deployment guides, and troubleshooting sections from major frameworks to identify frequently mentioned error scenarios. This provided 47 well-documented canonical errors.

Automated Error Injection (7.3%). We developed scripts to systematically inject realistic errors into working deployments, including dependency version mismatches, configuration parameter errors, missing environment variables, permission issues, and resource conflicts. This contributed 35 additional controlled scenarios.

This multi-source approach yielded **487 unique deployment scenarios**, ensuring comprehensive coverage of real-world deployment challenges.

4.3 Task Construction

4.3.1 Task Scenarios and Instructions. Each task in our benchmark consists of:

- **Natural language instruction:** A clear, concise description of the deployment goal, written in the style of GitHub issue reports or production incident tickets (e.g., "Deploy this Node.js application and ensure it runs on port 3000")
- **Installation/Deployment guide:** Step-by-step instructions for the intended deployment process, typically extracted from project README files, documentation, or deployment guides. This provides the agent with the canonical deployment procedure that should work but currently fails
- **Project files:** Complete application code, configuration files, and dependencies (provided via Git repository or archive), including package manifests, configuration templates, and build scripts
- **Error logs:** Initial error output from failed deployment attempts, capturing the failure state that agents must diagnose and resolve

- **Expected outcome:** Description of the successfully deployed state, including service availability requirements, expected responses, and verification criteria

Instructions are derived from real deployment scenarios and written in the style of GitHub issue reports or production incident tickets. We ensure instructions are unambiguous while avoiding giving away the solution directly.

4.3.2 Initial State Setup and Error Injection. To maximize realism, reproducibility, and controllability, each task includes a carefully crafted setup configuration that establishes the initial environment state and systematically injects realistic deployment errors:

- (1) **File preparation:** Project files are downloaded into a designated workspace directory (`~/workspace/`) with proper directory structure, mirroring fresh project clones from version control
- (2) **Controlled error injection:** Setup scripts systematically inject realistic errors that cause deployment failures. This is the *core mechanism* of our benchmark design. Injection strategies include:
 - *Dependency manipulation:* Modifying package manifests (`package.json`, `requirements.txt`, `pom.xml`) to introduce version conflicts or remove critical dependencies
 - *Configuration corruption:* Altering configuration files to contain invalid values, wrong paths, or missing required parameters
 - *Environment tampering:* Setting incorrect environment variables, removing required variables, or creating conflicting system configurations
 - *Permission modification:* Changing file/directory permissions to trigger access-denied errors during deployment
 - *Resource conflicts:* Pre-occupying ports, creating file locks, or consuming resources to simulate conflicts
- (3) **Tool availability:** All necessary diagnostic tools (logs, package managers, editors, system monitors) are made accessible, with proper permissions and `PATH` configurations

4.3.3 Execution-Based Evaluation. Unlike benchmarks that rely on predefined action sequences or manual verification, we implement **300 unique execution-based evaluation functions** that automatically assess task completion:

- to be continued

4.4 Benchmark Characteristics

4.4.1 Dataset Statistics. Table 1 presents comprehensive statistics of our benchmark. The dataset spans diverse technology stacks with realistic error distributions matching real-world deployment patterns.

4.4.2 Error Characteristics and Patterns. We analyze key characteristics that influence deployment difficulty:

Dependency Depth Distribution. Most errors occur at depth 2-4 (68.3% of cases), where indirect dependencies create conflicts. Shallow errors (depth 1, 21.4%) are typically straightforward missing

Table 1: Comprehensive statistics of our deployment benchmark.

Metric	Count
Total deployment scenarios	487
- Single-error cases	328 (67.3%)
- Multi-error cases	159 (32.7%)
Technology Stacks	
Docker/Containers	142 (29.2%)
Node.js/JavaScript	98 (20.1%)
Python/Django/Flask	87 (17.9%)
Java/Maven/Gradle	64 (13.1%)
Web Servers (Nginx/Apache)	43 (8.8%)
Databases (MySQL/PostgreSQL/MongoDB)	31 (6.4%)
System Configuration	22 (4.5%)
Error Categories	
Dependency conflicts	169 (34.7%)
Configuration errors	138 (28.3%)
Permission issues	77 (15.8%)
Environment setup failures	60 (12.4%)
Network/Resource problems	43 (8.8%)

packages, while deep dependency issues (depth 5+, 10.3%) require extensive resolution chains involving transitive dependencies.

Root Cause Categories. We classify scenarios by underlying causes: version mismatches (34.5%), missing configurations (26.7%), system-level issues like permissions and paths (18.9%), resource conflicts such as port collisions (12.3%), and documentation gaps representing undocumented requirements (7.6%).

Fix Pattern Distribution. Solutions fall into four categories: command execution only (42.1%) for package installation and service management, configuration file modification (31.8%) for editing JSON/YAML/XML files, multi-step sequential fixes (19.4%) requiring ordered resolution, and environment configuration (6.7%) for setting variables and creating files.

Log Characteristics. Error messages span 1-50 lines (median: 12), with varying clarity. We include explicit error messages (67.2%) with clear indicators and implicit failures (32.8%) requiring deeper investigation. Primary log sources are build logs (38.4%), runtime logs (49.9%), and system monitoring logs (11.7%).

4.5 Evaluation Metrics

We evaluate GUI agents on two primary dimensions: **task success rate** and **efficiency in terms of interaction steps**, both benchmarked against human performance to provide meaningful context.

4.5.1 Success Rate. The success rate measures the percentage of tasks an agent completes correctly according to the execution-based validation criteria (Section 4.3).

4.5.2 Interaction Efficiency (Step Count). To assess operational efficiency, we count the total number of atomic GUI or terminal interactions required to complete each task, including mouse clicks, keystrokes, command executions, and file edits.

5 EVALUATION

We evaluate Tool based on the following research questions.

- **RQ1. Effectiveness Evaluation.** How is the effectiveness of GUI Agents?

5.1 Setup

5.2 Setup

Dataset.

Baselines.

Metrics.

5.3 Discussion

Threats. First,

Limitations. First,

6 RELATED WORK

We review previous work related to

7 CONCLUSIONS

In this paper,

REFERENCES

349		407
350		408
351		409
352		410
353		411
354		412
355		413
356		414
357		415
358		416
359		417
360		418
361		419
362		420
363		421
364		422
365		423
366		424
367		425
368		426
369		427
370		428
371		429
372		430
373		431
374		432
375		433
376		434
377		435
378		436
379		437
380		438
381		439
382		440
383		441
384		442
385		443
386		444
387		445
388		446
389		447
390		448
391		449
392		450
393		451
394		452
395		453
396		454
397		455
398		456
399		457
400		458
401		459
402		460
403		461
404		462
405		463
406		464